

Module 4 R Notebook

Predicting Diseases from Genes

Important note - read first

In this course, you are not required to become proficient in writing R scripts for Big Biomedical Data Analytics. The assignments will be based on each module's template, and replacing some parts of them with new data and parameters so that you will get new results and interpret them.

Understanding all the coding details in R will depend on your background knowledge in programming concepts.

Therefore if you do not understand all the coding details in R, try to run the code and understand mostly:

- 1) what information do I have to provide to this script to make it work ?
- 2) how to execute this script (it will always be: run the cell containing the script Cell --> Run cells, or click on SHIFT + ENTER at the same time, or simply click on the right arrow icon) ?
- 3) what results do I get ?
- 4) how to interpret the results ?

Little by little, you will become familiar with R programming language and, example after example, you will understand the scripts and be ready for more advanced programming work.

Assignment goals

In this notebook, you are going to practice classification / prediction with R scripts, and particularly prediction based on gene expressions. This implies some data preprocessing then building a classification / prediction model to predict a disease (for example, this same process could be applied to different types of prediction).

Here we are continuing to work on a breast cancer dataset downloaded from Firehose (<https://gdac.broadinstitute.org/> (<https://gdac.broadinstitute.org/>)). These are next generation sequencing data that are provided already normalized.

We are going to compare two classifiers - KNN (K-nearest neighbor) and RandomForest (a classifier in the rule / tree family).

Each time we are going to build the models on the top 100 genes selected through bss/wss in Module 3, once using the training set as a test set (most favorable scenario), and next using the training set as 70% of the whole dataset, and the test set as 30% (least favorable scenario). We expect the classification accuracy to drop when moving from testing on the training set and training on 70% of the data. But how much is the accuracy going to drop ? And which of the two models is going to perform the best ? These are the questions we will try and answer.

So let us get started !

Preparing the environment

The processes employed to build the predictive model use quite a bit of RAM memory, therefore we first allocate this memory by increasaing the program memory size to 3.5GB (= 3,500 MB). We also load some libraries used for analysis - 'class' and 'randomForest'.

```
In [45]: # cell #1
memory.limit(size=3500)
library(randomForest)
library(class)
```

Out[45]: Inf

Loading the data

We again load the data as we did in module 3. We create a dataframe (a spreadsheet) 'mrnaNorm' with the gene expression values and the first column being the gene names. The second dataframe 'mrnaIDs' contains the IDs of the patients.

```
In [46]: # cell #2

mrnaNorm <- read.table("/data/BRCA.rnaseqv2__illuminahtseq_rnaseqv2__unc_edu__Lev
                  header = F, fill = T, skip = 2)
mrnaIDs <- read.table("/data/BRCA.rnaseqv2__illuminahtseq_rnaseqv2__unc_edu__Leve
                  header = F, fill = T, nrow = 1)
mrnaIDs <- mrnaIDs[, -1][, -1]
```

Data preprocessing

'mrnaClass' and 'mrnaClassNum' are created and contain the diagnostic class - 0 for normal and 1 for tumor.

```
In [47]: # cell #3
samp <- lapply(as.list(t(mrnaIDs)), function(t) substr(unlist(strsplit(t, "-"))[4
sampleType <- as.data.frame(samp)
sampClass <- lapply(samp, function(t) (if (t < 10) return("1") else return("0"))))
mrnaClass <- as.data.frame(sampClass)
dim(mrnaNorm)
# 20531 1213 columns are patients (except the 1st for gene name) rows are express
dim(mrnaIDs)
# 1 1213 the first column is the gene name, the others are one patient per row
dim(mrnaClass)
# 1 1212 one patients per row 1 = tumor, 0 = normal
table(unlist(sampClass))
# 0 1
# 112 1100 112 normals and 1100 tumor
sampClassNum <- lapply(samp, function(t) (if (t < 10) return(1) else return(0)))
mrnaClassNum <- as.data.frame(sampClassNum)
```

```
Out[47]: 20531 1213
```

```
Out[47]: 1 1212
```

```
Out[47]: 1 1212
```

```
Out[47]:
  0  1
112 1100
```

We also create a dataframe with only the gene names, called 'geneNames', which are located in the first column of 'mrnaNorm', which we extract.

```
In [48]: # cell #4
geneNames <- mrnaNorm[1] # extract the gene names from mrnaNorm as its first colu
dim(geneNames)
# 20531 genes
```

```
Out[48]: 20531 1
```

First we transpose the 'mrnaNorm' dataframe because we want to select genes, therefore they have to be represented in columns instead of rows. We also remove the first column of 'mrnaNorm' since it contains the gene names. Because we are working with large datasets, we free space from memory by removing the objects we will not be using anymore. 'gc' garbage collects the free space, which will leave more space for building the models. The column 'used' and right after it '(Mb)' indicates the memory in use.

```
In [49]: # cell #5
mrnaData = t(mrnaNorm[, -1]) # remove first column of mrnaData and transpose it t
rm(samp)
rm(sampClass)
rm(mrnaNorm)
gc()
```

Out[49]:

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	392528.0	21.0	5512073.0	294.4	12002346.0	641.0
Vcells	30115251.0	229.8	96267547.0	734.5	120329339.0	918.1

Classification / prediction on all features

It is possible to run the classification model on all 20,531 genes, however the processing lasts several minutes. For this reason, we are going to first select features before the classification to make it very efficient.

The code below runs KNN on all features, however I am going to skip it because it takes too long to run. If you are interested, you can test it. I have placed the '#' symbol in front of all the lines, you can remove it to run this cell (leave the one in front of the first line though).

```
In [50]: # cell #6
#trainSet <- mrnaData
#testSet <- mrnaData
#trainClasses <- unlist(mrnaClassNum[1,], use.names=FALSE)
#testClasses <- unlist(mrnaClassNum[1,], use.names=FALSE)
#knn.predic <- knn(trainSet, testSet, trainClasses, testClasses,k=1)
#cbr.predic = as.vector(knn.predic)
#table(cbr.predic, testClasses)
#tab <- table(cbr.predic, t(testClasses))
#error <- sum(tab) - sum(diag(tab))
#accuracy <- round(100- (error * 100 / length(testClasses)))
#print(paste("accuracy= ", as.character(accuracy), "%"), quote=FALSE)
```

Feature selection with bss/wss

Again, we select top genes with bss/wss. First we load the bssWssFast function.

```
In [51]: # cell #7
bssWssFast <- function (X, givenClassArr, numClass=2)
# between squares / within square feature selection
{
  classVec <- matrix(0, numClass, length(givenClassArr))
  for (k in 1:numClass) {
    temp <- rep(0, length(givenClassArr))
    temp[givenClassArr == (k - 1)] <- 1
    classVec[k, ] <- temp
  }
  classMeanArr <- rep(0, numClass)
  ratio <- rep(0, ncol(X))
  for (j in 1:ncol(X)) {
    overallMean <- sum(X[, j]) / length(X[, j])
    for (k in 1:numClass) {
      classMeanArr[k] <-
        sum(classVec[k, ] * X[, j]) / sum(classVec[k, ])
    }
    classMeanVec <- classMeanArr[givenClassArr + 1]
    bss <- sum((classMeanVec - overallMean)^2)
    wss <- sum((X[, j] - classMeanVec)^2)
    ratio[j] <- bss/wss
  }
  sort(ratio, decreasing = TRUE, index = TRUE)
}
```

Then we again run 'bssWssFast' on our large dataset to rank the features within and across classes. We work from mrnaData, which is the transposed data from 'mrnaNorm' dataframe because we want to select genes, therefore they have to be represented in columns instead of rows.

```
In [52]: # cell #8
# select features
dim(mrnaData)
# 1212 20531 matrix
dim(mrnaClass)
# 1 1212
dim(mrnaClassNum)
# 1 1212
dim(geneNames)
# 20531 genes
bss <- bssWssFast(mrnaData, t(mrnaClassNum), 2)
```

```
Out[52]:      1212  20531
```

```
Out[52]:      1   1212
```

```
Out[52]:      1   1212
```

```
Out[52]:    20531    1
```

Classification / prediction on selected features on training set

First we are going to build a classification model on the complete dataset as a training set, then test its performance on the same complete dataset. We call that testing on the training set. This is the easiest task for a model. It is as though we gave students the test in advance and tested them during an exam on the same tests they had studied.

We extract the top 100 gene expressions and place them in 'mrnaDataReduced'.

We create a 'trainSet' and 'testSet' which are identical because we are going to test the model on the training set.

We extract the 'trainClasses' and 'testClasses' from mrnaClassNum. Since classification / prediction is a supervised model, we give the models the classes we know for training, then use these classes for testing whether a model predicts the right class.

```
In [53]: # cell #9
mrnaDataReduced <- mrnaData[,bss$ix[1:100]]
dim(mrnaDataReduced)
# 1212 100
trainSet <- mrnaDataReduced
testSet <- mrnaDataReduced
trainClasses <- unlist(mrnaClassNum[1,], use.names=FALSE)
# or as.numeric(mrnaClassNum[1,])
testClasses <- unlist(mrnaClassNum[1,], use.names=FALSE)
```

```
Out[53]:      1212  100
```

KNN on selected features on training set

First, we are going to build a KNN model on this reduced dataset of 100 genes and 1212 patients / normals with 112 normals and 1100 tumor patients.

We pass to the 'knn' function, from the 'class' package, a training set, a test set, the list of classes for the training set, and the list of classes for the test set.

We then build the confusion matrix as:

```
testClasses
```

```
knn.predic 0 1 0 112 0 1 0 1100
```

which shows that from test classes of value 0, 112 are predicted as value 0, and from the test classes of value 1, 1100 are predicted as 1. This yields an accuracy of 100%, being calculated as $TP + TN / TP + FP + TN + FN = (1100 + 112) / (1100 + 0 + 112 + 0)$.

It cannot get better than that - very fast, perfect result.

```
In [54]: # cell #10
knn.predic <- knn(trainSet, testSet, trainClasses, testClasses,k=1) # knn form 'c'
knn.predic = as.vector(knn.predic) # change knn.predic to become a vector
table(knn.predic, testClasses) # build the confusion matrix
tab <- table(knn.predic, t(testClasses))
error <- sum(tab) - sum(diag(tab)) # calculate accuracy
accuracy <- round(100- (error * 100 / length(testClasses)))
print(paste("accuracy= ", as.character(accuracy), "%"), quote=FALSE) # display
```

```
Out[54]:      testClasses
knn.predic    0      1
           0  112     0
           1    0 1100

[1] accuracy= 100 %
```

RandomForest on selected features on training set

Now let us compare the same easy scenario with RandomForest from 'randomForest' package.

The 'rf' function, which builds the RandomForest, takes slightly different arguments than 'knn'.

First, instead of having separate gene expressions set and diagnostic class set, it takes only one dataframe. Therefore we concatenate with 'cbind' the gene expressions and the diagnostic class 0/1.

Next, the target class needs to have a name, which we choose as 'class'.

In addition, for classification, the target variable, here 'class', needs to be categorical, which is obtained by applying 'as.factor'.

Finally, there are two steps. First building the model with 'randomForest', then testing it on the test set with 'predict'.

We run, and bingo, we get another 100% accuracy.

So KNN and RandomForest are tied on the training set.

We are going to break the tie by using a different training set and test set. In other words, we are going to raise the bar in difficulty. It is like in a sport competition.

```

In [55]: # cell #11
trainSetClass <- as.data.frame(cbind(trainSet, t(mrnaClassNum[1,]))) # concatena
testSetClass <- as.data.frame(cbind(testSet, t(mrnaClassNum[1,]))) # concatena
colnames(trainSetClass)[101] <- "class" # give a name to the class column
#trainSetClass$class <- as.numeric(trainSetClass$class) # for regression
trainSetClass$class <- as.factor(trainSetClass$class) # for classification
class(trainSetClass$class) # should be factor or categorical for classificat
rf <- randomForest(class ~., trainSetClass,
                    ntree=100,
                    importance=T) # build randomForest classifier
colnames(testSetClass)[101] <- "class" # give a name to the class column
testSetClass$class <- as.factor(testSetClass$class) # for classification
rf.predic <- predict(rf, testSetClass) # test the randomForest built model on th
rf.predic = as.vector(rf.predic) # change rf.predic to become a vector
table(rf.predic, testClasses) # build the confusion matrix
tab <- table(rf.predic, t(testClasses))
error <- sum(tab) - sum(diag(tab)) # calculate acuracy
accuracy <- round(100- (error * 100 / length(testClasses)))
print(paste("accuracy= ", as.character(accuracy), "%"), quote=FALSE)

```

Out[55]: 'factor'

```

Out[55]:      testClasses
rf.predic    0    1
          0 112    0
          1   0 1100

[1] accuracy= 100 %

```

Classification / prediction on selected features on independent test set

In this experiment, we are first going to separate the dataset into two sets:

1) a training set composed of 70% of the data samples 2) a test set composed of 30% of the data samples.

After that, all the steps will be the same and we are going to compare classification accuracy between KNN and RandomForest.

To split the dataset, there is a 'sample' function in R which here will take 70% of the rows. The rest will be the test set.

Therefore our training set will have 848 rows, and our test set will have 364 rows.

```
In [56]: # cell #12
nbRows <- nrow(mrnaDataReduced)
set.seed(22) # seet random seed so that we always get same samples drawn -
trainRows <- sample(1:nbRows, .70*nbRows)
trainSet <- mrnaDataReduced[trainRows, ]
testSet <- mrnaDataReduced[-trainRows, ]
dim(trainSet)
dim(testSet)
```

```
Out[56]:      848  100
```

```
Out[56]:      364  100
```

KNN on selected features on independent test set

We run again KNN on the different training and test sets and obtain a classification accuracy of 99%. We have 5 errors on 364 data samples, which is a very high result. The exact figure is:

$360 / 364 = 98.90\%$

```
In [57]: # cell #13
trainClasses <- unlist(mrnaClassNum[1,trainRows], use.names=FALSE)
testClasses <- unlist(mrnaClassNum[1,-trainRows], use.names=FALSE)
knn.predic <- knn(trainSet, testSet, trainClasses, testClasses,k=1)
knn.predic = as.vector(knn.predic)
table(knn.predic, testClasses)
tab <- table(knn.predic, t(testClasses))
error <- sum(tab) - sum(diag(tab))
accuracy <- round(100- (error * 100 / length(testClasses)))
print(paste("accuracy= ", as.character(accuracy), "%"), quote=FALSE)
```

```
Out[57]:      testClasses
knn.predic   0    1
           0  36    3
           1   1 324
```

```
[1] accuracy=  99 %
```

RandomForest on selected features on independent test set

We run again RandomForest on the different training and test sets and obtain a classification accuracy of 99%. We have 3 errors on 364 data samples, which is a very high result. The exact figure is:

$361 / 364 = 99.18\%$

Therefore once more KNN and RandomForest are tied. However we notice that the errors are not exactly the same. RandomForest has no misclassified '1', while KNN has three '1' classified as '0'. This means than in three cases, patients with a tumor are classified as being normal, which is what is called a false negative. In a medical domain, this type of error is more severe than the false positive because some patients will think that they are fine and not seek care while they should seek care.

```

In [58]: # cell #14
trainSetClass <- as.data.frame(cbind(trainSet, t(mrnaClassNum[1,trainRows])))
testSetClass <- as.data.frame(cbind(testSet, t(mrnaClassNum[1,-trainRows])))
colnames(trainSetClass)[101] <- "class"
trainSetClass$class <- as.factor(trainSetClass$class) # for classification
class(trainSetClass$class)
# should be factor for classification
rf <- randomForest(class ~., trainSetClass,
                    ntree=100,
                    importance=T)
colnames(testSetClass)[101] <- "class"
testSetClass$class <- as.factor(testSetClass$class) # for classification
rf.predic <- predict(rf ,testSetClass)
rf.predic = as.vector(rf.predic)
table(rf.predic, testClasses)
tab <- table(rf.predic, t(testClasses))
error <- sum(tab) - sum(diag(tab))
accuracy <- round(100- (error * 100 / length(testClasses)))
print(paste("accuracy= ", as.character(accuracy), "%"), quote=FALSE)

```

Out[58]: 'factor'

```

Out[58]:      testClasses
rf.predic    0    1
           0 34    0
           1  3 327

[1] accuracy=  99 %

```

This is just a small example of what is possible to do. There are hundreds of classification / prediction models, and same for feature selection methods. By combining them, we obtained thousands of scenarios for building a classification / prediction model. The two models selected above are particularly efficient and effective as we could see.